HAUNTED HOSTEL GAME DEVELOPMENT USING PYTHON

AIM

To design and develop an interactive 2D horror maze game named **Haunted Hostel** using Python and the Pygame library. The player explores a dark hostel, collects three keys, avoids a pursuing ghost, uses a flashlight, and reaches the exit to escape. The game includes keyboard controls, collision detection, ghost path finding, win/loss states, and restart and quit options.

PROCEDURE

1. Import the required Python modules: math, random, sys, and pygame.
2. Initialize Pygame and create the game window with a resolution of 1000 × 700 pixels.
3. Define colours, tile dimensions, the hostel maze layout, wall locations, player start, ghost start, three key locations, and the exit.
4. Create helper functions for drawing text, converting between pixel positions and maze tiles, checking walls, and calculating a shortest path.
5. Initialize the game state with the player, ghost, keys, flashlight, status message, and timers.
6. Read keyboard input. Use W/A/S/D or arrow keys to move, F to toggle the flashlight, E to interact with the exit, R to restart after a win/loss, and ESC to quit.
7. Move the player while checking walls so the player cannot pass through them.
8. Detect when the player reaches a key and remove that key from the maze.
9. Use breadth-first search (BFS) to calculate a walkable route from the ghost to the player; move the ghost along that route.
10. Detect whether the ghost catches the player. If so, set the game state to lost.
11. Allow the player to escape only after collecting all three keys and interacting near the exit.
12. Draw the maze, keys, exit, ghost, player, flashlight darkness effect, and heads-up display on every frame.
13. Display the appropriate end screen and verify movement, key collection, exit interaction, ghost chase, restart, and quit controls.

PROGRAM CODE

```
import math
import random
import sys
import pygame

# Haunted Hostel: 3-level Python/Pygame horror maze
# Controls: WASD/Arrows move | E unlock/enter exit | F flashlight | R restart | ESC quit
pygame.init()
pygame.display.set_caption('Haunted Hostel - 3 Levels')
WIDTH, HEIGHT = 1000, 700
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()
FONT = pygame.font.SysFont('consolas', 22)
SMALL = pygame.font.SysFont('consolas', 16)
BIG = pygame.font.SysFont('consolas', 52)

BLACK = (8, 9, 15)
WALL = (48, 51, 65)
FLOOR = (36, 38, 52)
FLOOR2 = (43, 45, 61)
WHITE = (238, 238, 245)
RED = (220, 45, 60)
GREEN = (80, 225, 145)
GOLD = (255, 205, 65)
BLUE = (75, 175, 255)
PURPLE = (155, 85, 180)

TILE = 50
BASE_MAP = [
    '########################',
    '#.....#.......#....#',
    '#.....#.......#....#',
    '#.....#.......#....#',
    '#.....#.......#....#',
    '###.###.########.####',
```

```
    '#.....#.........#',
    '#.....#.........#',
    '#.....#.........#',
    '#.....#.........#',
    '####.####.........#',
    '#.................#',
    '#.................#',
    '#.................#',
    '########################',
]
ROWS, COLS = len(BASE_MAP), len(BASE_MAP[0])
MAP_W, MAP_H = COLS * TILE, ROWS * TILE
OFFSET_X = (WIDTH - MAP_W) // 2
OFFSET_Y = (HEIGHT - MAP_H) // 2
walls = {(c, r) for r, line in enumerate(BASE_MAP) for c, ch in enumerate(line)
if ch == '#'}

LEVELS = [
    {'keys': [(9, 3), (3, 8), (13, 8)], 'exit': (17, 13), 'ghost': (17, 2), 'name': 'Old
Dormitory'},
    {'keys': [(2, 2), (15, 7), (7, 12)], 'exit': (2, 12), 'ghost': (17, 12), 'name':
'Abandoned Wing'},
    {'keys': [(4, 3), (11, 6), (16, 11)], 'exit': (17, 2), 'ghost': (2, 12), 'name':
'Basement Escape'},
]

def tile_center(tile):
    c, r = tile
    return OFFSET_X + c * TILE + TILE // 2, OFFSET_Y + r * TILE + TILE // 2

def pixel_to_tile(pos):
    x, y = pos
    return int((x - OFFSET_X) // TILE), int((y - OFFSET_Y) // TILE)

def is_wall_tile(tile):
    c, r = tile
    return tile in walls or c < 0 or r < 0 or c >= COLS or r >= ROWS

def draw_text(text, x, y, color=WHITE, font=FONT, center=False):
```

```
    surf = font.render(text, True, color)
    rect = surf.get_rect()
    rect.center = (x, y) if center else rect.center
    if not center:
        rect.topleft = (x, y)
    screen.blit(surf, rect)

def shortest_path(start, goal):
    if start == goal:
        return [start]
    queue = [start]
    came = {start: None}
    for cur in queue:
        if cur == goal:
            break
        c, r = cur
        for nxt in ((c+1, r), (c-1, r), (c, r+1), (c, r-1)):
            if not is_wall_tile(nxt) and nxt not in came:
                came[nxt] = cur
                queue.append(nxt)
    if goal not in came:
        return []
    path, cur = [], goal
    while cur is not None:
        path.append(cur)
        cur = came[cur]
    return path[::-1]

def new_level(level_index):
    cfg = LEVELS[level_index]
    px, py = tile_center((1, 1))
    gx, gy = tile_center(cfg['ghost'])
    return {
        'level': level_index, 'player': [float(px), float(py)], 'ghost': [float(gx),
float(gy)],
        'keys': cfg['keys'][:], 'exit': cfg['exit'], 'flashlight': True, 'state': 'playing',
        'message': f"Level {level_index+1}: Find 3 keys and reach the exit!",
'message_timer': 4.0,
```

```
        'ghost_repath': 0.0, 'ghost_path': [], 'elapsed': 0.0, 'safe_timer': 2.5,
    }

game = new_level(0)

def move_circle(pos, dx, dy, radius=13):
    for axis, amount in ((0, dx), (1, dy)):
        if not amount:
            continue
        pos[axis] += amount
        pts = [(pos[0]-radius, pos[1]-radius), (pos[0]+radius, pos[1]-radius),
               (pos[0]-radius, pos[1]+radius), (pos[0]+radius, pos[1]+radius)]
        if any(is_wall_tile(pixel_to_tile(p)) for p in pts):
            pos[axis] -= amount

def draw_world():
    screen.fill(BLACK)
    for r in range(ROWS):
        for c in range(COLS):
            rect = pygame.Rect(OFFSET_X+c*TILE, OFFSET_Y+r*TILE, TILE,
TILE)
            if (c, r) in walls:
                pygame.draw.rect(screen, WALL, rect)
                pygame.draw.line(screen, (75, 78, 94), rect.topleft, rect.topright, 2)
                pygame.draw.line(screen, (20, 21, 30), rect.bottomleft,
rect.bottomright, 2)
            else:
                pygame.draw.rect(screen, FLOOR if (c+r)%2 == 0 else FLOOR2, rect)
                pygame.draw.rect(screen, (49, 51, 67), rect, 1)
    draw_text(LEVELS[game['level']]['name'].upper(), OFFSET_X+18,
OFFSET_Y+12, (150, 155, 180), SMALL)
    for key in game['keys']:
        x, y = tile_center(key)
        pygame.draw.circle(screen, GOLD, (x, y), 10)
        pygame.draw.circle(screen, (255, 240, 170), (x-2, y-3), 3)
        pygame.draw.rect(screen, GOLD, (x+5, y-2, 11, 4))
        pygame.draw.rect(screen, GOLD, (x+12, y-2, 3, 8))
    ex, ey = tile_center(game['exit'])
    door_color = GREEN if not game['keys'] else RED
```

```
    pygame.draw.rect(screen, door_color, (ex-17, ey-22, 34, 44), border_radius=4)
    pygame.draw.rect(screen, (245, 230, 190), (ex-11, ey-16, 22, 32), 2,
border_radius=3)
    pygame.draw.circle(screen, GOLD, (ex+8, ey), 3)
    gx, gy = map(int, game['ghost'])
    pygame.draw.circle(screen, PURPLE, (gx, gy), 18)
    pygame.draw.circle(screen, (235, 205, 245), (gx-6, gy-5), 5)
    pygame.draw.circle(screen, (235, 205, 245), (gx+6, gy-5), 5)
    pygame.draw.circle(screen, BLACK, (gx-6, gy-4), 2)
    pygame.draw.circle(screen, BLACK, (gx+6, gy-4), 2)
    px, py = map(int, game['player'])
    pygame.draw.circle(screen, BLUE, (px, py), 14)
    pygame.draw.circle(screen, (190, 230, 255), (px-4, py-5), 5)
    # Show a small interaction hint when near the exit.
    if math.dist(pixel_to_tile(game['player']), game['exit']) <= 2:
        hint = 'PRESS E TO ESCAPE' if not game['keys'] else f"EXIT LOCKED -
{len(game['keys'])} KEY(S) LEFT"
        draw_text(hint, WIDTH//2, HEIGHT-58, GREEN if not game['keys'] else
GOLD, SMALL, center=True)

def apply_darkness():
    # Brighter, larger flashlight so the maze and player are easy to see.
    overlay = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    if not game['flashlight']:
        overlay.fill((0, 0, 0, 150))
        screen.blit(overlay, (0, 0))
        return
    overlay.fill((0, 0, 0, 115))
    px, py = map(int, game['player'])
    pygame.draw.circle(overlay, (0, 0, 0, 35), (px, py), 205)
    pygame.draw.circle(overlay, (0, 0, 0, 18), (px, py), 175)
    pygame.draw.circle(overlay, (0, 0, 0, 0), (px, py), 155)
    screen.blit(overlay, (0, 0))

def draw_hud():
    panel = pygame.Surface((WIDTH, 52), pygame.SRCALPHA)
    panel.fill((0, 0, 0, 205))
    screen.blit(panel, (0, 0))
    draw_text('HAUNTED HOSTEL', 18, 14, (235, 215, 245), FONT)
```

```
    draw_text(f"LEVEL {game['level']+1}/3", 260, 16, BLUE)
    draw_text(f"KEYS: {3-len(game['keys'])}/3", 390, 16, GOLD)
    draw_text('WASD/ARROWS Move | E Door | F Light | ESC Quit', 535, 18,
WHITE, SMALL)
    if game['message_timer'] > 0 and game['message']:
        draw_text(game['message'], WIDTH//2, HEIGHT-30, (255, 235, 170),
SMALL, center=True)

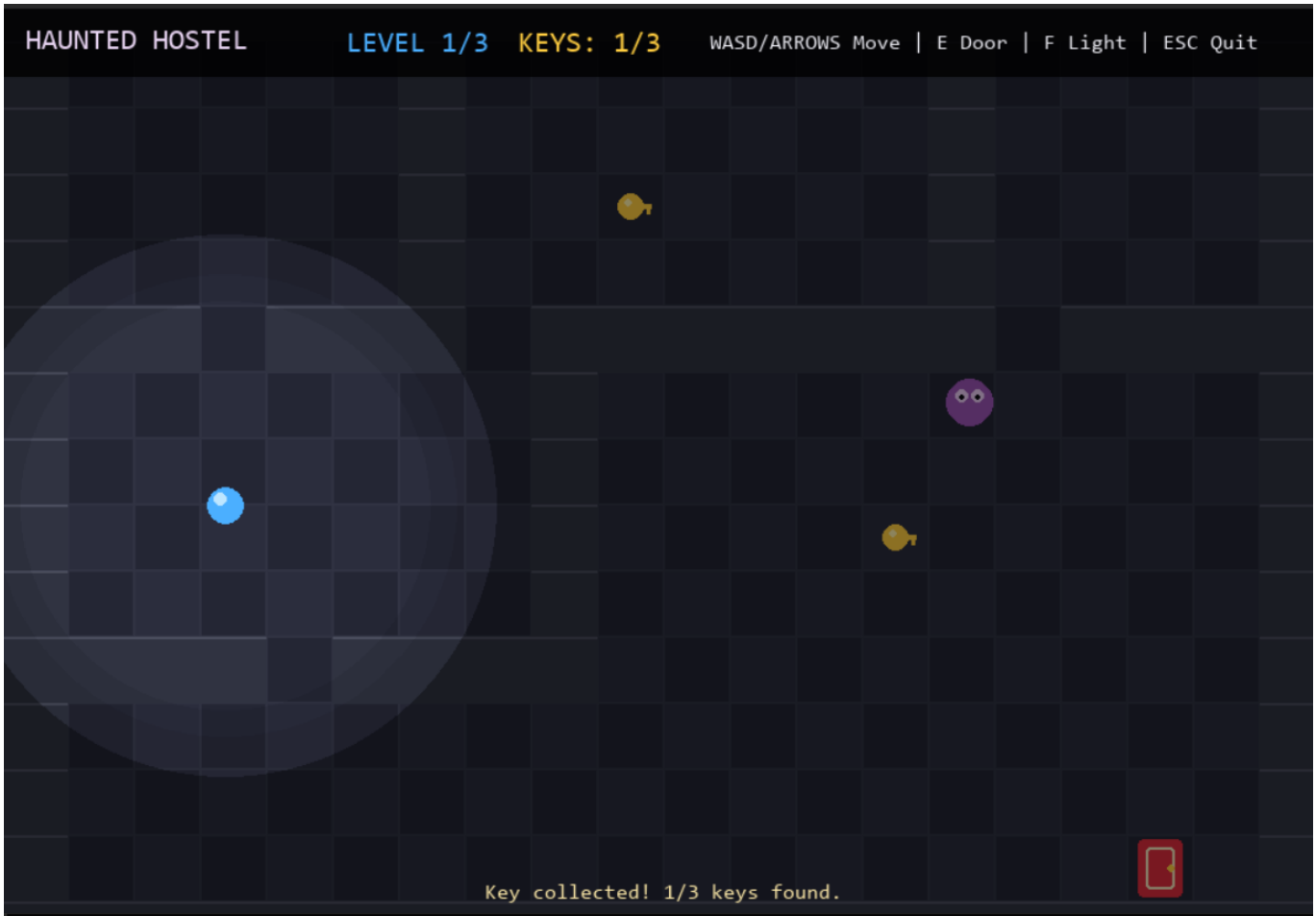
def draw_overlay(title, subtitle, color, extra='Press R to restart | ESC to quit'):
    shade = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    shade.fill((0, 0, 0, 205))
    screen.blit(shade, (0, 0))
    draw_text(title, WIDTH//2, HEIGHT//2-48, color, BIG, center=True)
    draw_text(subtitle, WIDTH//2, HEIGHT//2+15, WHITE, FONT,
center=True)
    draw_text(extra, WIDTH//2, HEIGHT//2+62, WHITE, SMALL, center=True)

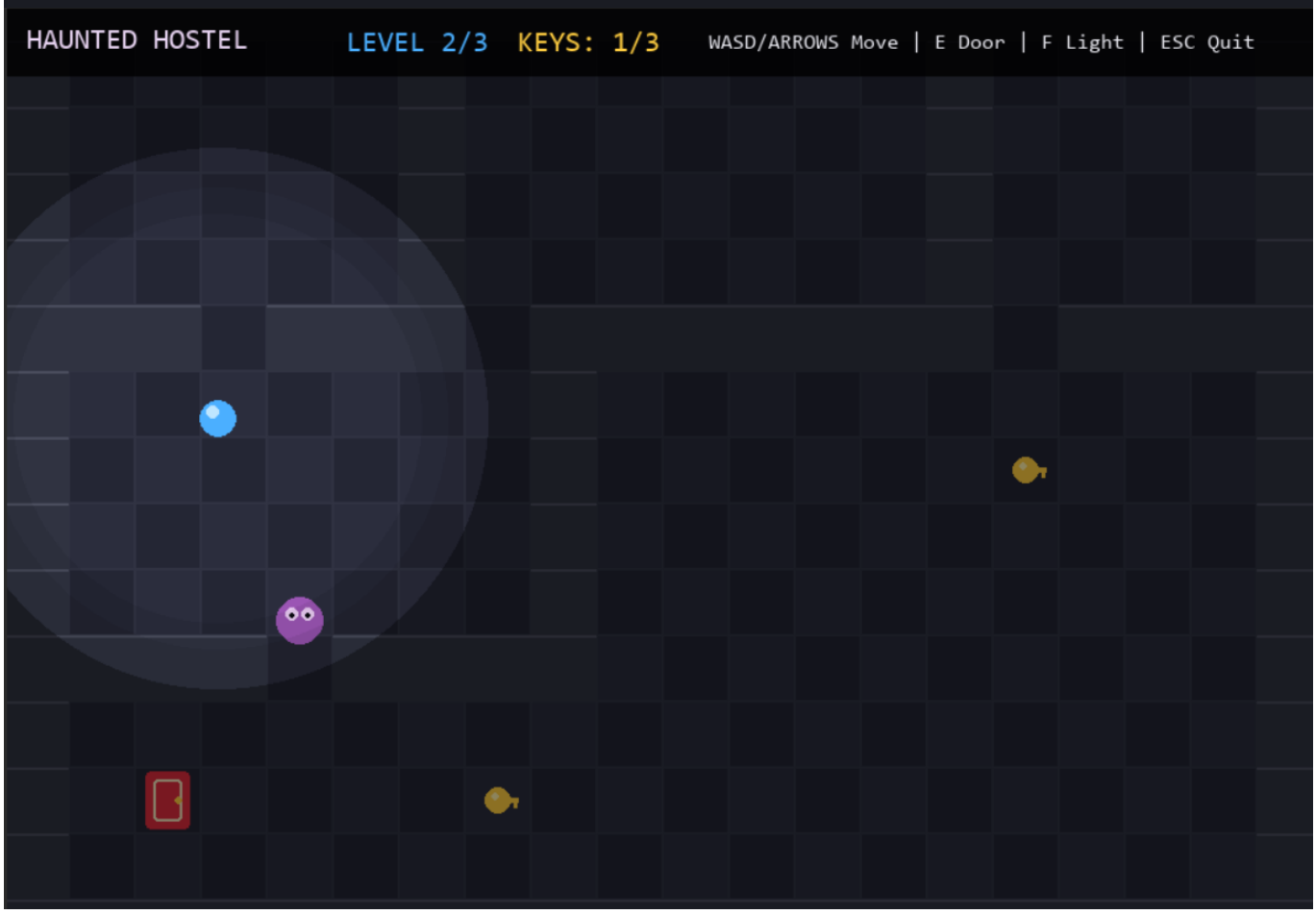
running = True
while running:
    dt = min(clock.tick(60) / 1000.0, 0.05)
    if game['state'] == 'playing':
        game['elapsed'] += dt
        game['safe_timer'] = max(0.0, game['safe_timer']-dt)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                running = False
            elif game['state'] != 'playing' and event.key == pygame.K_r:
                game = new_level(0)
            elif game['state'] == 'playing':
                if event.key == pygame.K_f:
                    game['flashlight'] = not game['flashlight']
                elif event.key == pygame.K_e:
                    # Use pixel distance so pressing E works reliably when standing
near the door.
                    if math.dist(game['player'], tile_center(game['exit'])) <= TILE *
1.65:
```

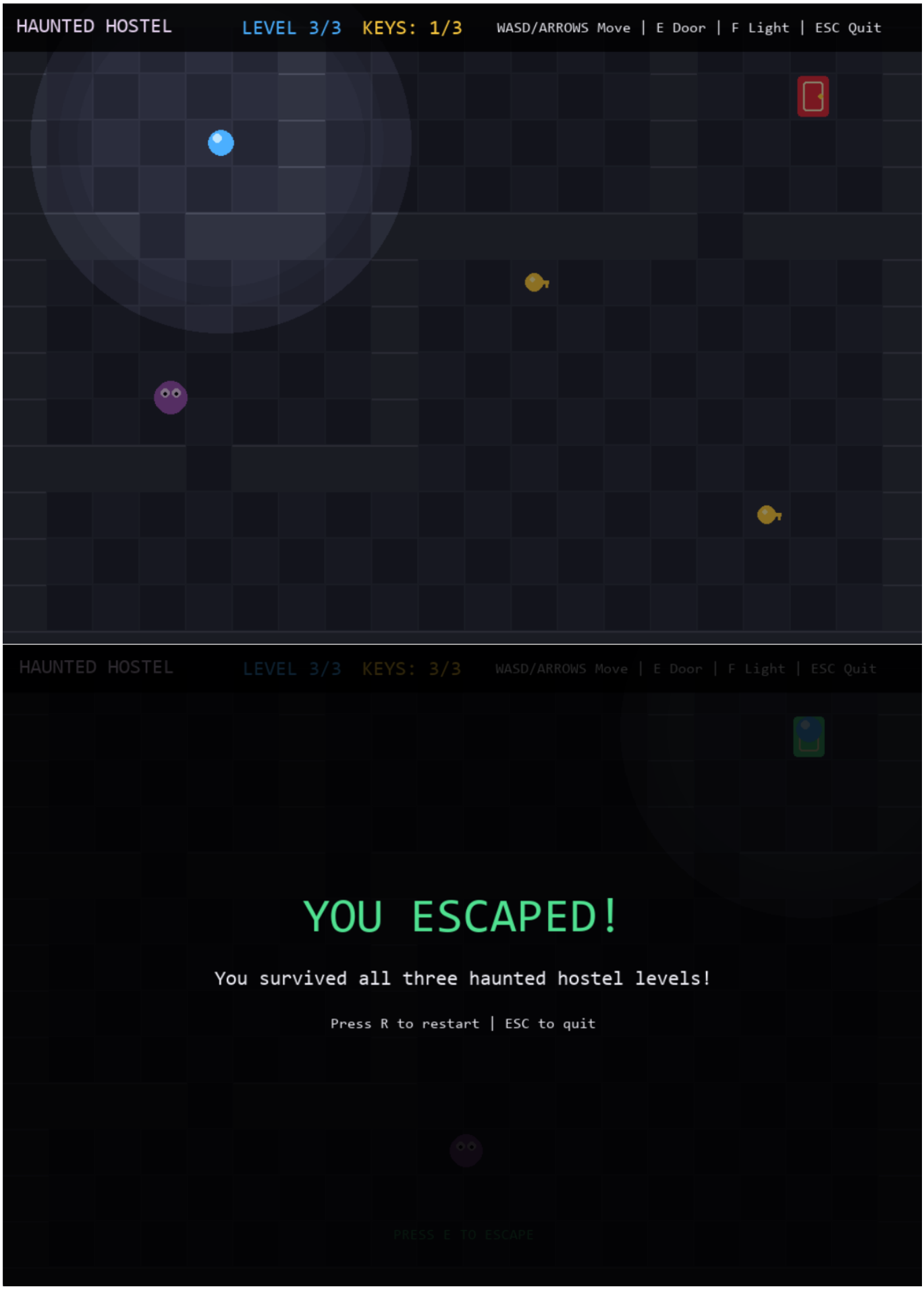
```
                if not game['keys']:
                    if game['level'] < 2:
                        game = new_level(game['level'] + 1)
                    else:
                        game['state'] = 'won'
                else:
                    game['message'] = f"Door locked! You need {len(game['keys'])}
more key(s)."
                    game['message_timer'] = 2.5
            else:
                game['message'] = 'Find the exit door first!'
                game['message_timer'] = 1.5
    if game['state'] == 'playing':
        pressed = pygame.key.get_pressed()
        dx = int(pressed[pygame.K_RIGHT] or pressed[pygame.K_d]) -
int(pressed[pygame.K_LEFT] or pressed[pygame.K_a])
        dy = int(pressed[pygame.K_DOWN] or pressed[pygame.K_s]) -
int(pressed[pygame.K_UP] or pressed[pygame.K_w])
        length = math.hypot(dx, dy)
        if length:
            move_circle(game['player'], dx/length * 185 * dt, dy/length * 185 * dt)
        player_tile = pixel_to_tile(game['player'])
        for key in game['keys'][:]:
            if math.dist(player_tile, key) <= 0.8:
                game['keys'].remove(key)
                game['message'] = f"Key collected! {3-len(game['keys'])}/3 keys found."
                game['message_timer'] = 1.8
        game['ghost_repath'] -= dt
        if game['ghost_repath'] <= 0:
            game['ghost_repath'] = 0.55
            game['ghost_path'] = shortest_path(pixel_to_tile(game['ghost']),
player_tile)
        path = game['ghost_path']
        if len(path) > 1:
            tx, ty = tile_center(path[1])
            gx, gy = game['ghost']
            vx, vy = tx-gx, ty-gy
            dist = math.hypot(vx, vy)
            if dist > 2:
```

```
            # Ghost starts slowly and gets faster with each level, but remains
avoidable.
            speed = (34 + game['level']*10) + min(game['elapsed']*0.25, 8)
            game['ghost'][0] += vx/dist * speed * dt
            game['ghost'][1] += vy/dist * speed * dt
        if game['safe_timer'] <= 0 and math.dist(game['player'], game['ghost']) <
25:
            game['state'] = 'lost'
        if game['message_timer'] > 0:
            game['message_timer'] -= dt
    draw_world()
    apply_darkness()
    draw_hud()
    if game['state'] == 'won':
        draw_overlay('YOU ESCAPED!', 'You survived all three haunted hostel
levels!', GREEN)
    elif game['state'] == 'lost':
        draw_overlay('CAUGHT!', 'The ghost caught you. Try moving and use the
brighter light.', RED)
    pygame.display.flip()
pygame.quit()
sys.exit()
```

OUTPUT


HAUNTED HOSTEL
LEVEL 1/3
KEYS: 1/3
WASD/ARROWS Move | E Door | F Light | ESC Quit
Key collected! 1/3 keys found.


HAUNTED HOSTEL
LEVEL 2/3
KEYS: 1/3
WASD/ARROWS Move | E Door | F Light | ESC Quit


HAUNTED HOSTEL
LEVEL 3/3
KEYS: 1/3
WASD/ARROWS Move | E Door | F Light | ESC Quit
HAUNTED HOSTEL
LEVEL 3/3
KEYS: 3/3
WASD/ARROWS Move | E Door | F Light | ESC Quit
YOU ESCAPED!
You survived all three haunted hostel levels!
Press R to restart | ESC to quit

RESULT

Thus, the Haunted Hostel 2D horror maze game was developed using Python and Pygame. The project demonstrates graphical game rendering, keyboard-based movement, maze collision detection, key collection, flashlight effects, ghost pursuit using breadth-first search, win/loss handling, and restart and exit controls.